

PV284 Colloquium Project

Patrik Adam

24.1.2025

1 Project brief description

This project implements an infrared (IR) remote control system for a computer, allowing wireless control of selected functions as well as physical power on/off operation. An Arduino-based device receives signals from a standard IR remote, decodes them, and sends corresponding commands to a computer application via a serial connection, where they are translated into keyboard actions. A micro servo motor is used to mechanically press the computer's power button, while LEDs provide system status indication. Main components:

- Arduino-compatible board with ATmega2560 microcontroller
- HX1838 IR receiver module
- SG90 micro servo motor
- Samsung TV IR remote control
- Laptop computer
- Breadboard and connecting wires
- Status LEDs
- 9V battery

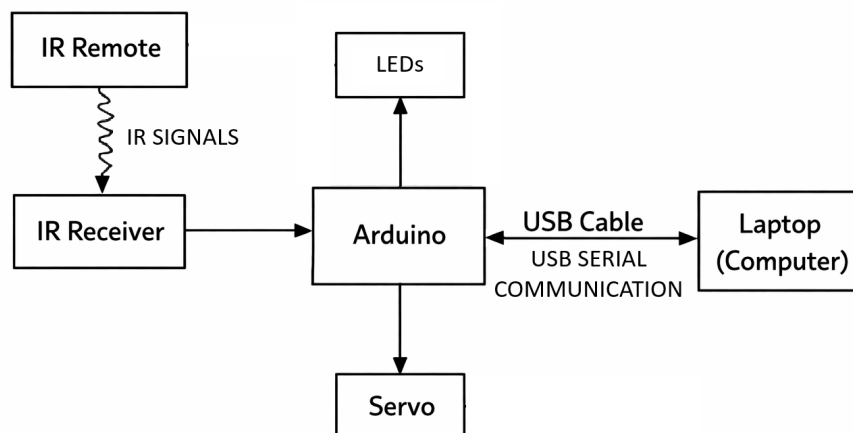


Figure 1: Remote controller - sketch

2 Project solution

The system operates as an interface between an infrared remote control and a personal computer.

An HX1838 infrared receiver detects IR signals transmitted by a consumer remote control. These signals are read and decoded by an Arduino-compatible board based on the ATmega2560 microcontroller. The Arduino software filters the received data and processes only predefined, relevant command codes, ignoring all other IR signals.

When a designated power-on command is received, the Arduino controls an SG90 micro servo motor to mechanically press the computer's power button. After initiating the power sequence, the Arduino attempts to establish communication with the computer over a USB virtual serial (COM) port. A handshake mechanism is implemented in which the Arduino periodically transmits a PING message and waits for a PONG response from the computer application.

Once communication is established, the Arduino continuously listens for IR commands. For each valid command, it sends a corresponding text-based instruction through the serial connection. The Arduino also implements a keep-alive mechanism, periodically sending test messages to verify that the connection remains functional during operation.

LEDs are used for indicating the state of the controller running on Arduino - off, pairing, connected or error.

On the computer side, a custom application monitors the serial port, responds to handshake and keep-alive messages, and interprets incoming command strings. Using the Windows API, specifically functions for input simulation, the application generates virtual keyboard events. These include multimedia controls (volume up/down, play/pause, next/previous track), navigation keys (up, down, left, right), confirmation keys (Enter, Escape), and system-level shortcuts such as Alt+Tab for switching between active windows.

3 Conclusion

The developed system represents a functional working prototype of an infrared remote control interface for a personal computer. The solution successfully achieves its intended goals: wireless control of computer functions and the ability to physically power the machine on and off using a servo-driven mechanism. Communication between the Arduino-based hardware and the computer application over a USB virtual serial (COM) port proved reliable, and the translation of remote control signals into simulated keyboard inputs operated as expected.

During development, several technical challenges had to be addressed. One significant issue was that the Arduino automatically resets when the serial port is opened by the computer, even when the board is powered through a different supply. This behavior required adjustments in the communication initialization process. Another limitation encountered was related to power delivery: the SG90 servo motor did not operate reliably when the Arduino was powered solely from the computer's USB port, indicating insufficient current capability. Additionally, powering the system from a standard 9 V battery was found to be unsuitable for long-term operation due to limited capacity and voltage stability.

Despite these challenges, the system performs its intended functions. However, several improvements could enhance reliability and functionality. First, replacing the Arduino board with a model that supports USB HID (Human Interface Device) functionality would allow the device to act directly as a keyboard, enabling control even before user login on the computer. Second, implementing a power-saving mode for the Arduino when the computer is off would reduce unnecessary energy consumption, leaving only the IR reception active. Third, upgrading the actuation mechanism—either by using a higher-quality servo or replacing it with a solenoid—could improve the mechanical reliability of the power-button pressing system.

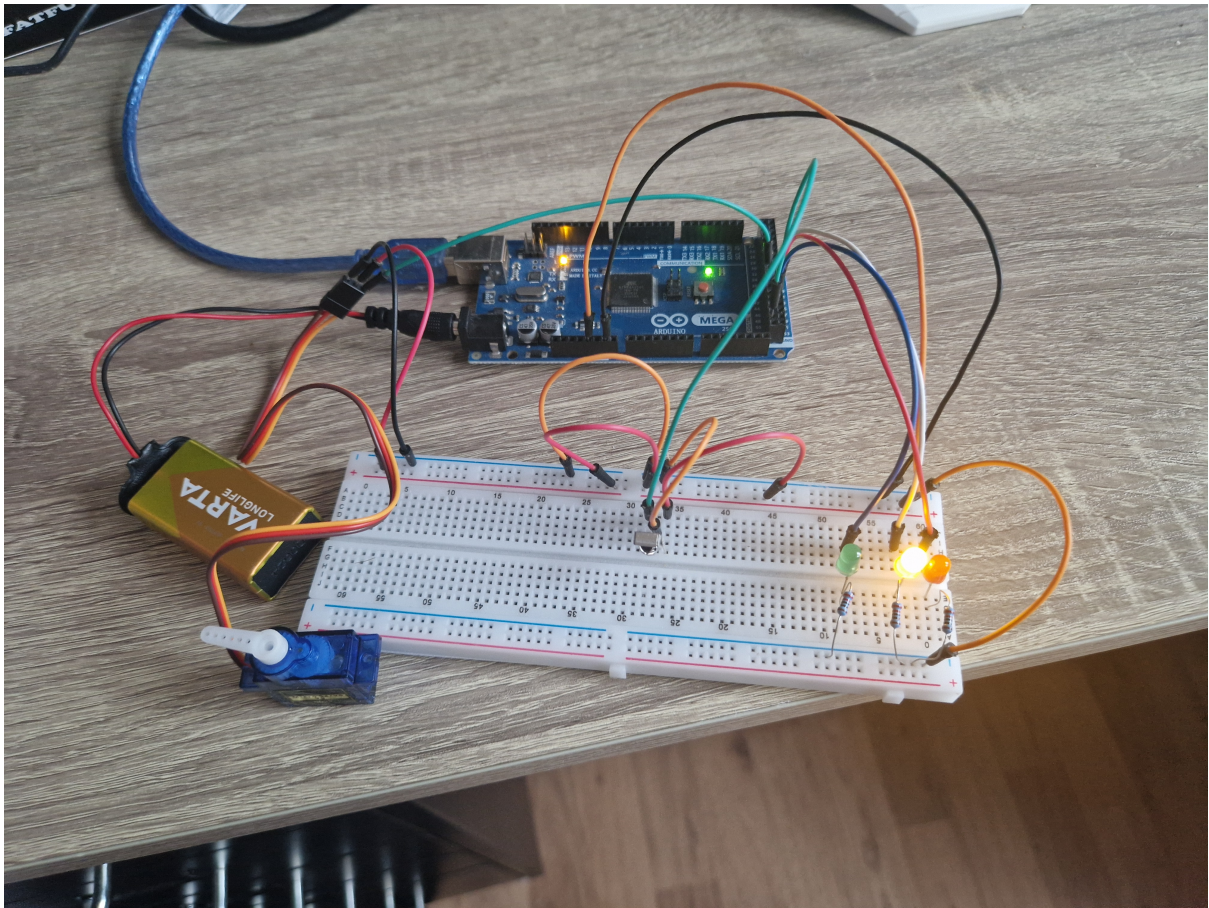


Figure 2: Photo of remote controller

Overall, the project demonstrates a practical and extensible approach to integrating consumer IR control with computer hardware and software systems.

4 References - if needed

IR receiver datasheet:

https://www.laskakit.cz/user/related_files/vs1838b.pdf

Arduino Mega2560 clone:

ATMEL ATmega2560 datasheet from <https://www.laskakit.cz>

Servo SG90 9g:

https://www.kjell.com/globalassets/mediaassets/701916_87897_datasheet_en.pdf

Arduino libraries:

<https://docs.arduino.cc/libraries/servo/>

<https://github.com/Arduino-IRremote/Arduino-IRremote/tree/master>

5 Appendixes

5.1 Software of Arduino node

The software for ATmega2560 was prepared on Arduino IDE. The source code is here:

5.1.1 Main .ino file

```
// main .ino file
#include <Servo.h>
#include <Arduino.h>
#include "PinDefinitionsAndMore.h"
#include <IRremote.hpp>
#include "LedController.h"
#include "SerialProtocol.h"
```

```
#define DECODE_SAMSUNG
```

```
Servo myservo;
const int servoPin = 36;
const int greenLedPin = 47;
const int yellowLedPin = 45;
const int redLedPin = 43;
```

```
Led greenLed;
Led yellowLed;
Led redLed;
```

```
enum SystemState {
    OFF,
    PAIRING,
    CONNECTED,
```

```

    ERROR
};

SystemState state = PAIRING;

void setup() {
    ledInit(greenLed, 47);
    ledInit(yellowLed, 45);
    ledInit(redLed, 43);

    myservo.attach(servoPin);

    Serial.begin(9600);
    IrReceiver.begin(IR_RECEIVE_PIN, ENABLE_LED_FEEDBACK);
}

void loop() {
    ledUpdate(greenLed);
    ledUpdate(yellowLed);
    ledUpdate(redLed);

    switch (state) {
        case PAIRING:
            ledOn(yellowLed);
            if (serialInitHandshake()) {
                state = CONNECTED;
            } else {
                state = OFF;
            }
            ledOff(yellowLed);
            break;

        case OFF:
            if (IrReceiver.decode()) {
                if (!(IrReceiver.decodedIRData.flags & IRDATA_FLAGS_IS_REPEAT)) {
                    uint8_t cmd = IrReceiver.decodedIRData.command;
                    if (cmd == 0x02) {
                        servo_push();
                        state = PAIRING;
                    }
                }
                IrReceiver.resume();
            }
            break;

        case CONNECTED:
            ledOn(greenLed);
            if (IrReceiver.decode()) {
                if (!(IrReceiver.decodedIRData.flags & IRDATA_FLAGS_IS_REPEAT)) {
                    uint8_t cmd = IrReceiver.decodedIRData.command;
                    if (cmd == 0x02) {

```

```

        servo_push();
        ledPulse(redLed, 300);
        ledOff(greenLed);
        state = OFF;
    } else {
        serialSendCommand(cmd);
    }
}
IrReceiver.resume();
}
if (!serialAliveCheck()) {
    state = ERROR;
    ledOff(greenLed);
}
break;

case ERROR:
    ledPulse(redLed, 3600);
    state = OFF;
    break;

default:
    state = OFF;
}
}

void servo_push(void) {
    myservo.write(90);
    delay(150);
    myservo.write(120);
    delay(150);
    myservo.write(90);
}

```

5.1.2 PinDefinitionsAndMore.h

Part of IRremote library example:

<https://github.com/Arduino-IRremote/.../PinDefinitionsAndMore.h>

5.1.3 LedController.h

```
#ifndef LED_CONTROLLER_H
#define LED_CONTROLLER_H

#include <Arduino.h>

enum LedMode {
    LED_OFF,
    LED_ON,
    LED_PULSE,
    LED_BLINK
};

struct Led {
    uint8_t pin;
    LedMode mode;
    bool state;
    unsigned long interval;
    unsigned long lastChange;
};

/* Initialization */
void ledInit(Led &led, uint8_t pin);

/* Update (call every loop) */
void ledUpdate(Led &led);

/* Control API */
void ledOn(Led &led);
void ledOff(Led &led);
void ledPulse(Led &led, unsigned long durationMs);
void ledBlink(Led &led, unsigned long periodMs);

#endif // LED_CONTROLLER_H
```

5.1.4 LedController.cpp

```
#include "LedController.h"

void ledInit(Led &led, uint8_t pin) {
    led.pin = pin;
    led.mode = LED_OFF;
    led.state = LOW;
    led.interval = 0;
    led.lastChange = 0;

    pinMode(pin, OUTPUT);
    digitalWrite(pin, LOW);
}

void ledUpdate(Led &led) {
    unsigned long now = millis();

    switch (led.mode) {

        case LED_OFF:
            digitalWrite(led.pin, LOW);
            led.state = LOW;
            break;

        case LED_ON:
            digitalWrite(led.pin, HIGH);
            led.state = HIGH;
            break;

        case LED_PULSE:
            if (!led.state) {
                digitalWrite(led.pin, HIGH);
                led.state = HIGH;
                led.lastChange = now;
            }
            else if (now - led.lastChange >= led.interval) {
                digitalWrite(led.pin, LOW);
                led.state = LOW;
                led.mode = LED_OFF;
            }
            break;

        case LED_BLINK:
            if (now - led.lastChange >= led.interval) {
                led.state = !led.state;
                digitalWrite(led.pin, led.state);
                led.lastChange = now;
            }
            break;
    }
}
```

```

void ledOn(Led &led) {
    led.mode = LED_ON;
    ledUpdate(led);
}

void ledOff(Led &led) {
    led.mode = LED_OFF;
    ledUpdate(led);
}

void ledPulse(Led &led, unsigned long durationMs) {
    led.mode = LED_PULSE;
    led.interval = durationMs;
    led.state = LOW;
}

void ledBlink(Led &led, unsigned long periodMs) {
    led.mode = LED_BLINK;
    led.interval = periodMs;
    led.lastChange = millis();
}

```

5.1.5 SerialProtocol.h

```
#ifndef SERIAL_PROTOCOL_H
#define SERIAL_PROTOCOL_H

#include <Arduino.h>

/* ===== Configuration ===== */
#define SERIAL_BAUDRATE          9600
#define PING_INTERVAL_MS        1000UL    // how often to send PING
#define INIT_TIMEOUT_MS         36000UL    // 3 minutes
#define ALIVE_INTERVAL_MS       20000UL    // periodic alive check
#define PONG_TIMEOUT_MS         5000UL     // wait for PONG

/* ===== API ===== */
void serialProtocolInit();
bool serialWaitForPong(unsigned long timeoutMs);
bool serialInitHandshake();
bool serialAliveCheck();
void serialSendCommand(uint8_t cmd);

#endif
```

5.1.6 SerialProtocol.cpp

```
#include "SerialProtocol.h"

/* ===== Internal state ===== */
static unsigned long lastAliveCheck = 0;

/* ===== Command mapping ===== */
static const char* commandToString(uint8_t cmd) {
    switch (cmd) {
        case 0x12:            return "ALT_TAB";
        case 0x10:            return "ALT_SHIFT_TAB";
        case 0x65:            return "LEFT";
        case 0x62:            return "RIGHT";
        case 0x60:            return "UP";
        case 0x61:            return "DOWN";
        case 0x68:            return "ENTER";
        case 0x47:            return "PLAY";
        case 0x4A:            return "PAUSE";
        case 0x48:            return "NEXT";
        case 0x45:            return "PREVIOUS";
        case 0x7:             return "VOLUME_UP";
        case 0xB:             return "VOLUME_DOWN";
        case 0xF:             return "MUTE";
        case 0x58:            return "RETURN";
        case 0x2D:            return "ESCAPE";
        default:              return "UNKNOWN";
    }
}

/* ===== Initialization ===== */
void serialProtocolInit() {
    Serial.begin(SERIAL_BAUDRATE);
    while (!Serial) {
        ; // wait for serial (important for Leonardo, harmless for UNO)
    }
}

/* ===== Low-level PONG wait ===== */
bool serialWaitForPong(unsigned long timeoutMs) {
    unsigned long start = millis();
    String line;

    while (millis() - start < timeoutMs) {
        while (Serial.available()) {
            char c = Serial.read();
            if (c == '\n') {
                line.trim();
                if (line == "PONG") {
                    return true;
                }
                line = "";
            }
        }
    }
}
```

```

        } else {
            line += c;
        }
    }
}
return false;
}

/* ===== 1. Initialization handshake ===== */
bool serialInitHandshake() {
    unsigned long start = millis();

    while (millis() - start < INIT_TIMEOUT_MS) {

        Serial.println("PING");

        if (serialWaitForPong(PONG_TIMEOUT_MS)) {
            return true; // success
        }

        delay(PING_INTERVAL_MS);
    }

    // Timeout reached
    return false;
}

/* ===== 2. Periodic alive check ===== */
bool serialAliveCheck() {
    unsigned long now = millis();

    if (now - lastAliveCheck < ALIVE_INTERVAL_MS) {
        return true;
    }

    lastAliveCheck = now;
    Serial.println("PING");

    if (!serialWaitForPong(PONG_TIMEOUT_MS)) {
        // optional: handle lost connection
        return false;
    }
    return true;
}

/* ===== 3. Send command ===== */
void serialSendCommand(uint8_t cmd) {
    if (commandToString(cmd) != "UNKNOWN") {
        Serial.println(commandToString(cmd));
    }
}

```

5.2 Software of computer node

Single file keyboard_controller.cpp, for demonstration and debugging purposes it prints received commands on standard output.

5.2.1 keyboard_controller.cpp

```
#include <windows.h>
#include <iostream>
#include <string>

// ----- SERIAL HELPERS -----

HANDLE openSerial(const char* portName) {
    HANDLE hSerial = CreateFileA(
        portName,
        GENERIC_READ | GENERIC_WRITE,
        0,
        nullptr,
        OPEN_EXISTING,
        0,
        nullptr
    );

    if (hSerial == INVALID_HANDLE_VALUE) {
        std::cerr << "Failed to open serial port\n";
        return INVALID_HANDLE_VALUE;
    }

    DCB dcb{};
    dcb.DCBlength = sizeof(dcb);
    GetCommState(hSerial, &dcb);

    dcb.BaudRate = CBR_9600;      // Must match Arduino
    dcb.ByteSize = 8;
    dcb.Parity = NOPARITY;
    dcb.StopBits = ONESTOPBIT;

    SetCommState(hSerial, &dcb);

    COMMTIMEOUTS timeouts{};
    timeouts.ReadIntervalTimeout = 50;
    timeouts.ReadTotalTimeoutConstant = 50;
    timeouts.ReadTotalTimeoutMultiplier = 10;
    SetCommTimeouts(hSerial, &timeouts);

    return hSerial;
}

void writeLine(HANDLE hSerial, const std::string& s) {
    DWORD written;
    WriteFile(hSerial, s.c_str(), s.size(), &written, nullptr);
}
```

```

}

// ----- KEYBOARD HELPERS -----

void keyPress(WORD vk) {
    INPUT input{};
    input.type = INPUT_KEYBOARD;
    input.ki.wVk = vk;
    SendInput(1, &input, sizeof(INPUT));
}

void keyRelease(WORD vk) {
    INPUT input{};
    input.type = INPUT_KEYBOARD;
    input.ki.wVk = vk;
    input.ki.dwFlags = KEYEVENTF_KEYUP;
    SendInput(1, &input, sizeof(INPUT));
}

void keyTap(WORD vk) {
    keyPress(vk);
    Sleep(20);
    keyRelease(vk);
}

void altTab() {
    keyPress(VK_MENU);    // ALT
    keyTap(VK_TAB);
    keyRelease(VK_MENU);
}

// ----- COMMAND HANDLER -----

void handleCommand(const std::string& cmd) {
    if (cmd == "UP") {
        keyTap(VK_UP);
    }
    else if (cmd == "DOWN") {
        keyTap(VK_DOWN);
    }
    else if (cmd == "LEFT") {
        keyTap(VK_LEFT);
    }
    else if (cmd == "RIGHT") {
        keyTap(VK_RIGHT);
    }
    else if (cmd == "ALT_TAB") {
        altTab();
    }
    else if (cmd == "ALT_SHIFT_TAB") {
        keyPress(VK_MENU);    // ALT
    }
}

```

```

        keyPress(VK_SHIFT); // SHIFT
        keyTap(VK_TAB);      // TAB
        keyRelease(VK_SHIFT);
        keyRelease(VK_MENU);
    }
    else if (cmd == "ENTER") {
        keyTap(VK_RETURN);
    }
    else if (cmd == "ESCAPE") {
        keyTap(VK_ESCAPE);
    }
    else if (cmd == "PLAY" || cmd == "PAUSE") {
        keyTap(VK_MEDIA_PLAY_PAUSE);
    }
    else if (cmd == "VOLUME_UP") {
        keyTap(VK_VOLUME_UP);
    }
    else if (cmd == "VOLUME_DOWN") {
        keyTap(VK_VOLUME_DOWN);
    }
    else if (cmd == "MUTE") {
        keyTap(VK_VOLUME_MUTE);
    }
    else if (cmd == "RETURN") {
        keyPress(VK_MENU); // ALT
        keyTap(VK_LEFT);  // LEFT
        keyRelease(VK_MENU);
    }
    else if (cmd == "PREVIOUS") {
        keyTap(VK_MEDIA_PREV_TRACK);
    }
    else if (cmd == "NEXT") {
        keyTap(VK_MEDIA_NEXT_TRACK);
    }
    else {
        std::cout << "Unknown command: " << cmd << "\n";
    }
}

// ----- MAIN LOOP -----

int main() {
    HANDLE serial = openSerial("COM3"); // <-- CHANGE THIS
    if (serial == INVALID_HANDLE_VALUE) return 1;

    std::string buffer;
    char ch;
    DWORD read;

    std::cout << "Listening...\n";

```

```

while (true) {
    if (ReadFile(serial, &ch, 1, &read, nullptr) && read == 1) {
        if (ch == '\n') {
            // Remove CR if present
            if (!buffer.empty() && buffer.back() == '\r')
                buffer.pop_back();

            std::cout << "RX: " << buffer << "\n";

            if (buffer == "PING") {
                writeLine(serial, "PONG\n");
            } else {
                handleCommand(buffer);
            }

            buffer.clear();
        } else {
            buffer += ch;
        }
    }
}

CloseHandle(serial);
return 0;
}

```